

Microservice Communication

 systemdesignschool.io/fundamentals/microservice-communications



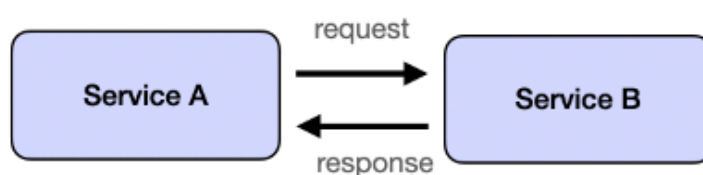
We explored microservices architecture and how it breaks applications into smaller, independent services. Each service handles a specific function with its own database. But services cannot work in isolation. When a user places an order, the order service must check inventory, process payment, and send notifications. How do these separate services coordinate?

The Communication Problem

An e-commerce site runs three microservices: orders, inventory, and payments. A customer buys a product. The order service receives the request. It needs inventory data to verify stock availability. It needs the payment service to charge the customer. Without communication between these services, the order cannot complete.

Synchronous Communication

Synchronous communication uses direct request-response interactions. Service A sends a request to Service B and waits for a response before continuing. The calling thread blocks during this time.



REST over HTTP is the most common implementation. The order service makes an HTTP GET request to the inventory service. The inventory service queries its database and returns JSON with stock levels. gRPC offers a more efficient alternative using HTTP/2 and Protocol Buffers.

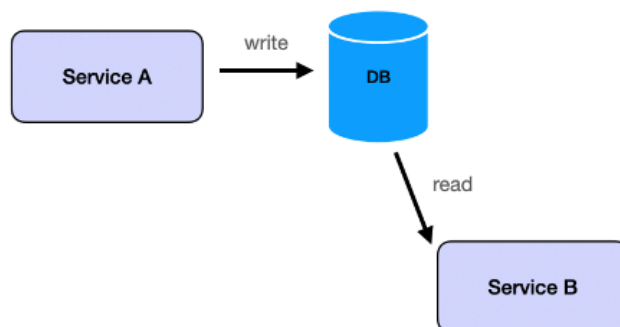
This pattern is simple to implement and debug. Code reads sequentially. Errors surface immediately. HTTP status codes indicate success or failure. Tools like curl and Postman make testing straightforward.

But synchronous calls create tight coupling. If the inventory service is down, the order fails completely. The user cannot place an order even though the order and payment services work fine. Failures cascade through service chains. Latency adds up across multiple hops.

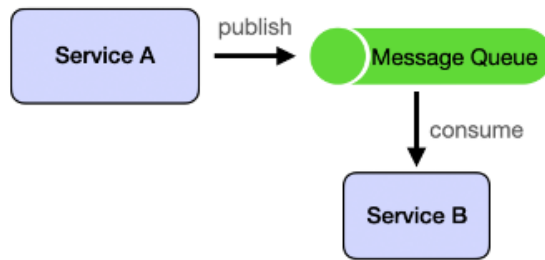
Asynchronous Communication

Asynchronous communication decouples services. The sender publishes a message and continues without waiting for a response. The receiver processes the message later.

One approach uses shared databases. Service A writes data to a table. Service B polls the table and processes new records. This is simple but creates tight coupling to the database schema. Both services must agree on table structure. Database becomes a bottleneck under high load.



Message queues provide a better solution. Service A publishes a message to RabbitMQ or Kafka. The queue guarantees delivery to Service B even if Service B is temporarily offline. Service B processes messages at its own pace.



When a user places an order, the order service publishes an "OrderCreated" event. The inventory service subscribes to this event and reserves stock. The notification service also subscribes and sends a confirmation email. Services work independently. If email fails, the order still completes.

Asynchronous communication handles high throughput well. Services do not block waiting for responses. The queue buffers messages during traffic spikes. But the complexity increases. Debugging distributed async flows is harder than tracing synchronous request chains.

Choosing a Pattern

Use synchronous communication when you need an immediate response. User authentication must validate credentials before granting access. Fetching data to display on screen requires the actual data now. Payment processing needs confirmation before completing checkout.

Use asynchronous communication for operations that can happen eventually. Sending notification emails does not need to block the user. Updating analytics dashboards can happen in the background. Processing uploaded files can take minutes without blocking the upload response.

Many systems use both patterns. A REST API handles user requests synchronously. Background jobs use message queues for async processing. The API gateway returns a response immediately while publishing events for downstream processing.

The following sections explore synchronous communication in depth, covering implementation with REST and gRPC, failure handling patterns, and service discovery.